



SQLMap – Automatizando a Detecção de SQL Injection

Seminário · Segurança e Teste de Software

Fábio Terra e Felipe Barroso

SQL Injection – O Problema Crítico

SQL Injection (SQLi) é uma vulnerabilidade de segurança que permite a um atacante manipular consultas a bancos de dados, inserindo código SQL malicioso através de campos de entrada da aplicação.

Impacto Devastador

Compromete a **confidencialidade** (acesso a dados sensíveis), **integridade** (modificação/exclusão de dados) e **disponibilidade** (negação de serviço).

Código Vulnerável

Aplicações que não validam corretamente as entradas do usuário ou não utilizam declarações preparadas são suscetíveis a ataques.

Testes Manuais Ineficientes

A detecção manual de SQLi é demorada, propensa a erros e não escalável para grandes bases de código.

```
10
11
12 function removeLinkHeader
13   return prev.then($ => {
14     $(':header').map((i, header) => {
15       const children = $(header).children();
16       if (children.length > 0) {
17         $(header).text('');
18         $(children).remove();
19       }
20       return header;
21     });
22   return Promise.resolve();
23 });
24 }
```

O que é o SQLMap?

Definição

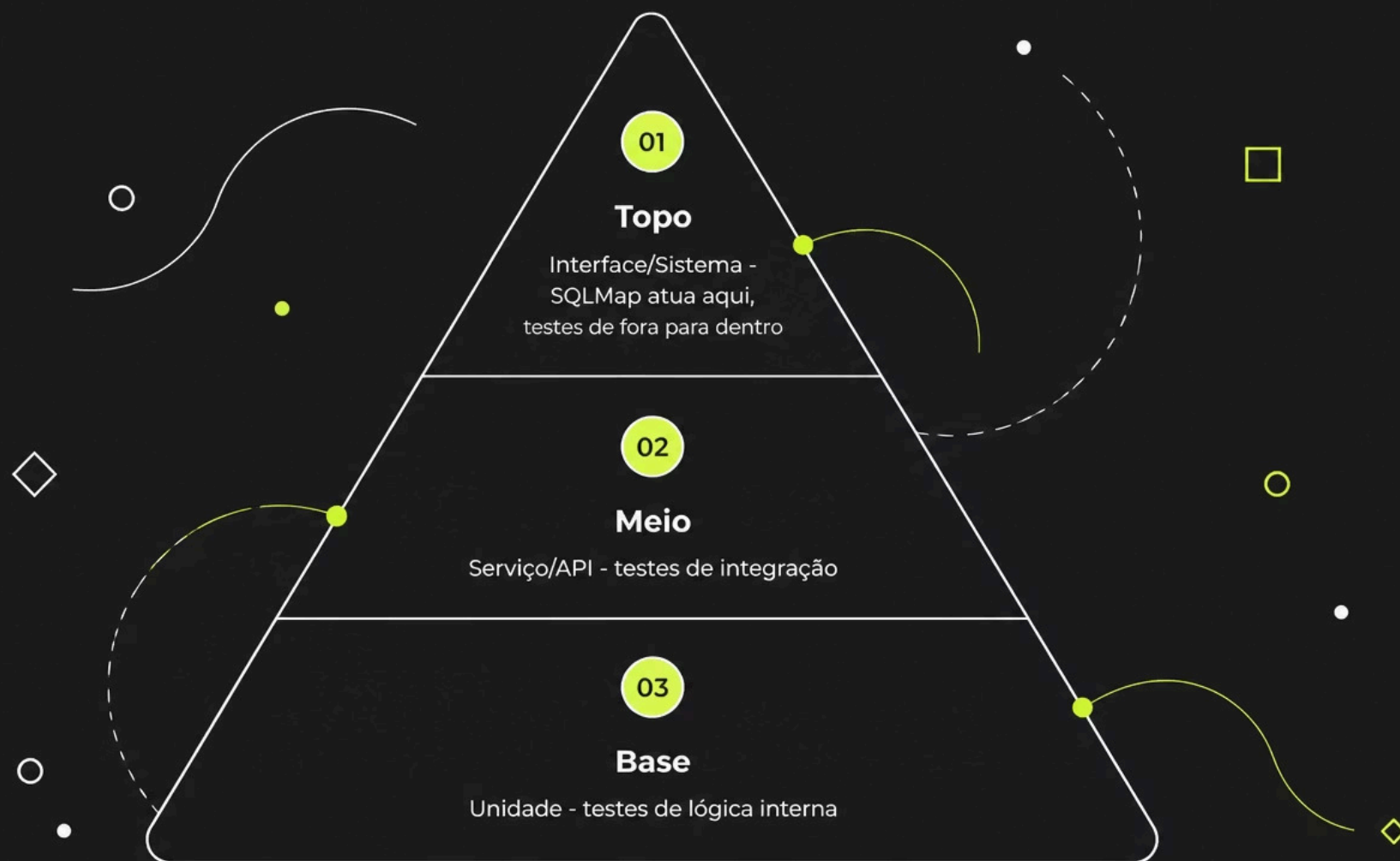
SQLMap é uma ferramenta **open-source em Python** que automatiza a detecção e exploração de vulnerabilidades de SQL Injection. Em vez de testar payloads manualmente, aponta-se para um alvo e ele identifica o ponto vulnerável, faz fingerprint do banco e extrai dados.

Repositório oficial: github.com/sqlmapproject/sqlmap

Posição na Pirâmide de Testes

O SQLMap atua na **camada de interface/sistema** (topo da pirâmide) — testa a aplicação de fora para dentro, pela mesma entrada que um usuário ou atacante usaria.

- Complementa testes de unidade e de API
- Onde eles verificam lógica interna, o SQLMap verifica o **comportamento real da aplicação exposta**



Principais Funcionalidades

Detecção Automática

SQLi em parâmetros GET, POST, cookies e headers.

Fingerprint de MySQL, PostgreSQL, MSSQL, Oracle, SQLite e outros.

Enumeração & Dump

Bancos, tabelas, colunas, usuários.

Dump de dados com quebra de hashes de senha via dicionário.

Acesso ao S0

Execução de comandos no servidor via `--os-shell` e `--sql-shell`. Leitura e escrita de arquivos no servidor de banco.

Bypass de WAF/IDS

Scripts de tamper para contornar firewalls de aplicação.

Teste no modo **caixa-preta**: apenas URL e parâmetros expostos.

Integrações

CI/CD: chamado via script em pipelines (GitHub Actions, GitLab CI, Jenkins) para testes de regressão. **Burp Suite**: aceita requisições salvas (`-r request.txt`). **Docker**: imagens prontas disponíveis. Sem integração nativa com JUnit/pytest — resultados via saída de texto/log.

Ataque por Nível de Segurança

O SQLMap adapta automaticamente a técnica ao nível de proteção do alvo:

1

LOW – GET sem proteção

```
sqlmap -u "http://localhost/vulnerabilities/sqli/?  
id=1&Submit=Submit" \  
--cookie="security=low; PHPSESSID=" \  
--batch --dbs
```

→ Lista bancos; dump extrai usuários e quebra hashes MD5.

2

MEDIUM – POST + mysqli_real_escape_string

```
sqlmap -u "http://localhost/vulnerabilities/sqli/" \  
--cookie="security=medium; PHPSESSID=" \  
--data="id=1&Submit=Submit" --batch --dbs
```

→ mysqli_real_escape_string não protege injeção numérica. Técnica muda para POST numérico.

3

HIGH – Injeção de Segundo Grau (Second-Order)

```
sqlmap -u ".../sqli/session-input.php" \  
--data="id=1&Submit=Submit" \  
--second-url="http://localhost/vulnerabilities/sqli/" \  
--batch --dbs
```

→ Ainda vulnerável; exige --second-url para ler resposta na página correta.

4

IMPOSSIBLE – Prepared Statement

```
sqlmap -u "..." --cookie="security=impossible" --batch --dbs
```

→ Ataque bloqueado. Nenhuma vulnerabilidade encontrada. Query parametrizada é a defesa eficaz.

Demonstração – Setup e Contexto

Repositório e Instalação

Repositório oficial: github.com/sqlmapproject/sqlmap

Instalação: `pip install sqlmap` ou clone do repositório.

Versão usada na demo: **versão estável do projeto**, instalada localmente para execução em ambiente controlado.

Makefile – Automação da Demo

- `make up`: inicia o DVWA em Docker.
- `make setup`: configura ambiente e dependências.
- `make scan`: executa SQLMap nos 4 níveis.
- `make crack`: quebra hashes de senha.
- `make level`: alterna entre níveis de segurança.
- `make down`: para os containers.

Ambiente de Teste

A demonstração utiliza o **DVWA (Damn Vulnerable Web Application)** rodando em `http://localhost`, em um ambiente próprio e autorizado para fins educacionais.

 **Aviso ético:** nunca execute testes de segurança sem autorização explícita do proprietário do sistema.

Frameworks Similares

Ferramentas alternativas que atuam no mesmo nível de teste de segurança de aplicações web:

Ferramenta	Foco Principal	Tipo de Teste
SQLMap	SQLi automatizado, mais completo e especializado	Caixa-preta
Havij	SQLi automatizado (Windows, GUI)	Caixa-preta
BBQSQL	SQLi blind, altamente customizável	Caixa-preta
Burp Suite Pro	Scanner web completo (inclui SQLi)	Caixa-preta
OWASP ZAP	Scanner DAST open-source	Caixa-preta
Nessus / Nikto	Scanner de vulnerabilidades mais amplo	Caixa-preta

❗ **Destaque:** O SQLMap é o mais especializado e completo para SQLi especificamente, com suporte a mais técnicas e DBMSs do que qualquer alternativa open-source.

Vantagens e Desvantagens

Vantagens

- Open-source e ativamente mantido no GitHub com atualizações frequentes
- Suporta **6 técnicas de injeção (BEUSTQ)** e mais de 30 DBMSs
- Altamente configurável: `--level`, `--risk`, `--tamper`, `--technique`
- Comunidade grande e documentação extensa
- Curva de entrada baixa para casos simples com `--batch` e `--wizard`

⚠ Desvantagens

- Pode gerar **falsos positivos** em aplicações com respostas inconsistentes
- Sem integração nativa com frameworks de teste (JUnit, pytest)
- **Uso indevido é crime** — requer contexto autorizado
- Para WAF customizado e tokens CSRF dinâmicos, a curva de aprendizado sobe consideravelmente
- Sem interface gráfica oficial — linha de comando pura

Casos de Sucesso



Bug Bounty

Amplamente usado em plataformas como **HackerOne** e **Bugcrowd** por pesquisadores para validar SQLi antes de reportar vulnerabilidades.



Pentesters Profissionais

Ferramenta padrão em avaliações de segurança autorizadas. Amplamente usada por pesquisadores em programas de bug bounty de empresas como **Yahoo** e **Microsoft**, bem como em testes de penetração profissionais.



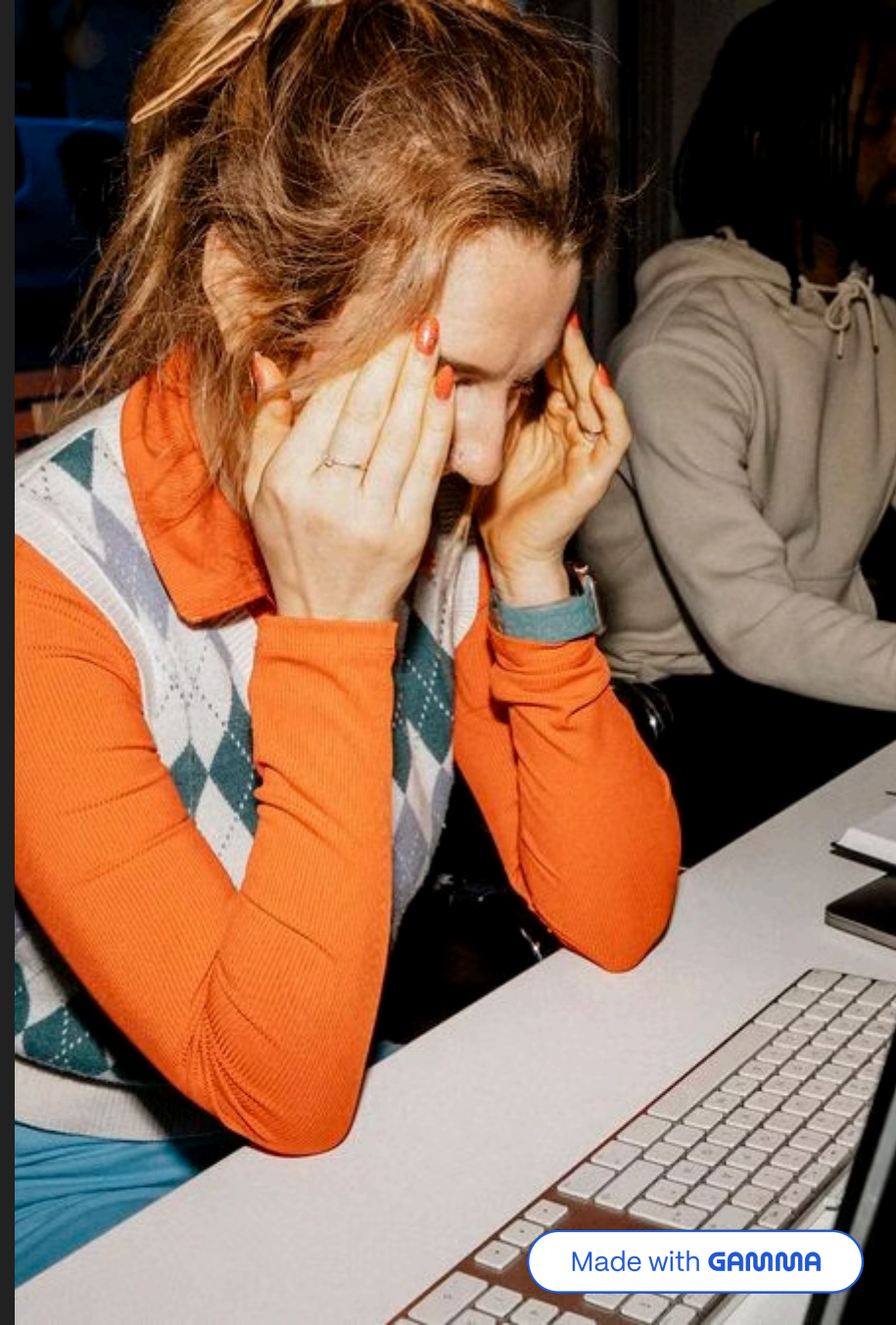
DevSecOps

Integrado em pipelines CI/CD para detectar regressões de SQLi antes do deploy em produção.



Referência OWASP

Referenciado pelo **OWASP Testing Guide** como ferramenta recomendada para testes de SQL Injection.



Conclusão

Avaliação Crítica

O SQLMap é **referência para testes de SQLi** — maduro, confiável e poderoso. A demonstração mostrou que aumentar a segurança não elimina a vulnerabilidade automaticamente: cada nível exigiu uma técnica diferente do atacante.

- ✓ **A única defesa que bloqueou o ataque foi o prepared statement (Impossible).**

Quando Adotar

- Testes de penetração autorizados em aplicações web
- Auditorias de segurança antes de deploys críticos
- Ambientes de aprendizado (DVWA, Juice Shop)

Quando NÃO Adotar

- Sem autorização explícita — uso não autorizado é crime
- Como substituto de SAST ou code review
- Para cobertura ampla de vulnerabilidades web (prefira ZAP ou Burp Suite)